

[Programmation] cardealer & javaFX

tp8

TP8 : Car Dealer & JAVA FX P1



Un garage souhaite gérer ses véhicules, ses vendeurs, les ventes et ses clients.

Votre application devra être en anglais.

Pour un véhicule, on souhaite avoir les propriétés suivantes : le nom du modèle, le constructeur, l'année de fabrication, les km parcourus, la condition : neuve ou occasion, le nombre pièces disponible, le prix, la devise, la disponibilité et stocker une image du véhicule.

Pour chaque client, on souhaite connaître : son nom, son téléphone, son email et son adresse.

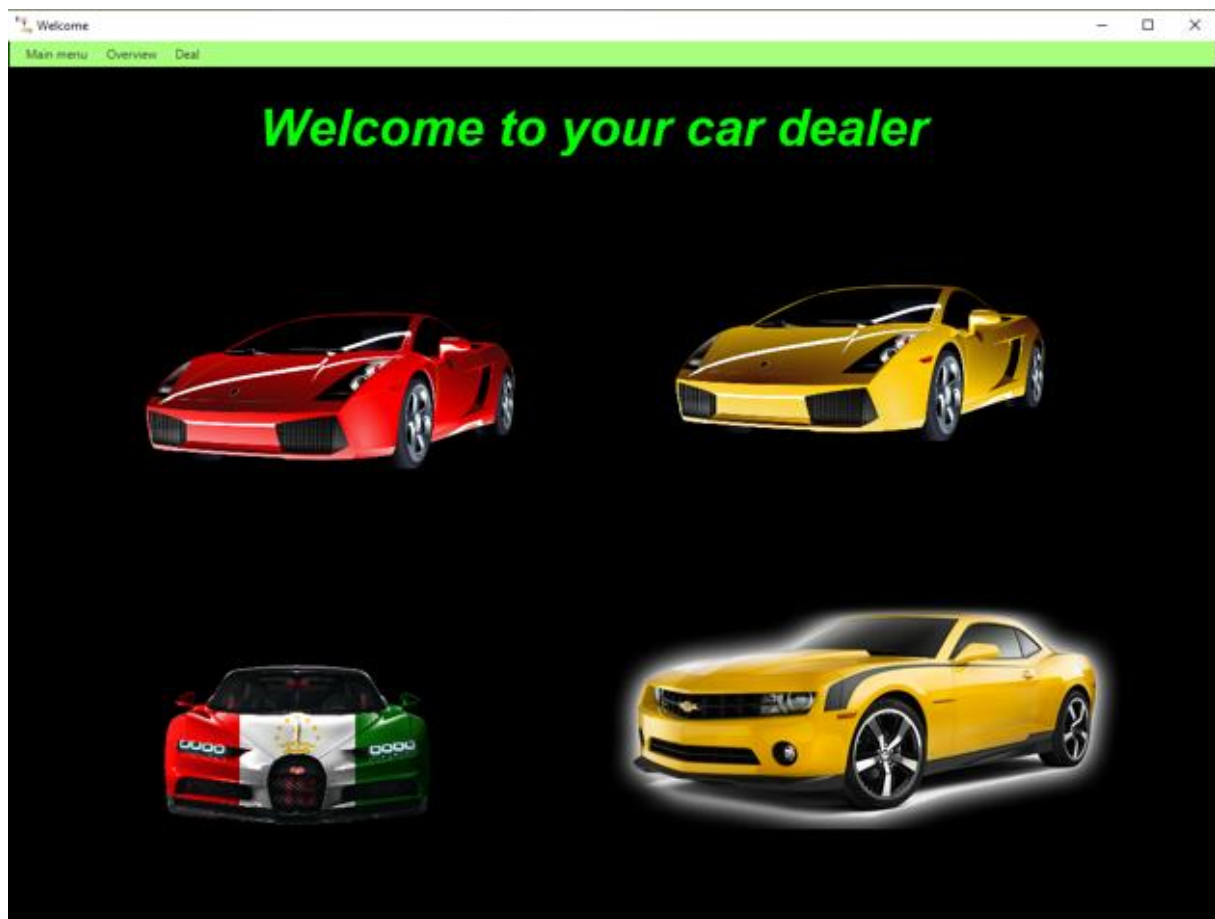
Pour chaque vendeur, on souhaite connaître les mêmes caractéristiques qu'un client.

Question 1 : Faire le diagramme de classe, générer le code java et les 4 tables dans Mysql.

Question 2 : créer un projet javafx Cardealer avec l'open jdk 22 puis implémenter les 3 classes, générer automatique le constructeur, les getters et les setters.

Le code est à placer dans le dossier src<java<com.example.cardealer. L'interface graphique est gérée avec la classe helloApplication qui lance votre fenêtre principale stockée dans le fichier hello-view.fxml présent dans le répertoire ressources<com.example.cardealer (vos fichiers FXML vont devoir être stocké dans ressources. Pour générer des événements via cette fenêtre principale, vous utiliserez HelloController par la suite.

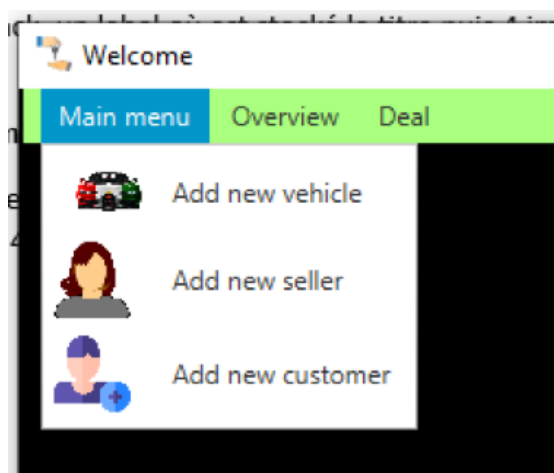
Question 3 : Créer la fenêtre principale de votre application dans le fichier hello-view.fxml puis scene Builder. Elle doit ressembler à cet écran :



Votre fenêtre principale mesure width 1200 et height 900, comprend un AnchorPane en background color black, un label où est stocké le titre puis 4 imagesview qui comprend 4 images au format w :500 h :250

Pour implémenter, l'image et le nom de la fenêtre, cela se situe sur votre fichier helloApplication avec les fonctions : `stage.setTitle("Votre-titre");` et `stage.getIcons().add(new Image("votre-image"));`

Question 4 : créer le menubar

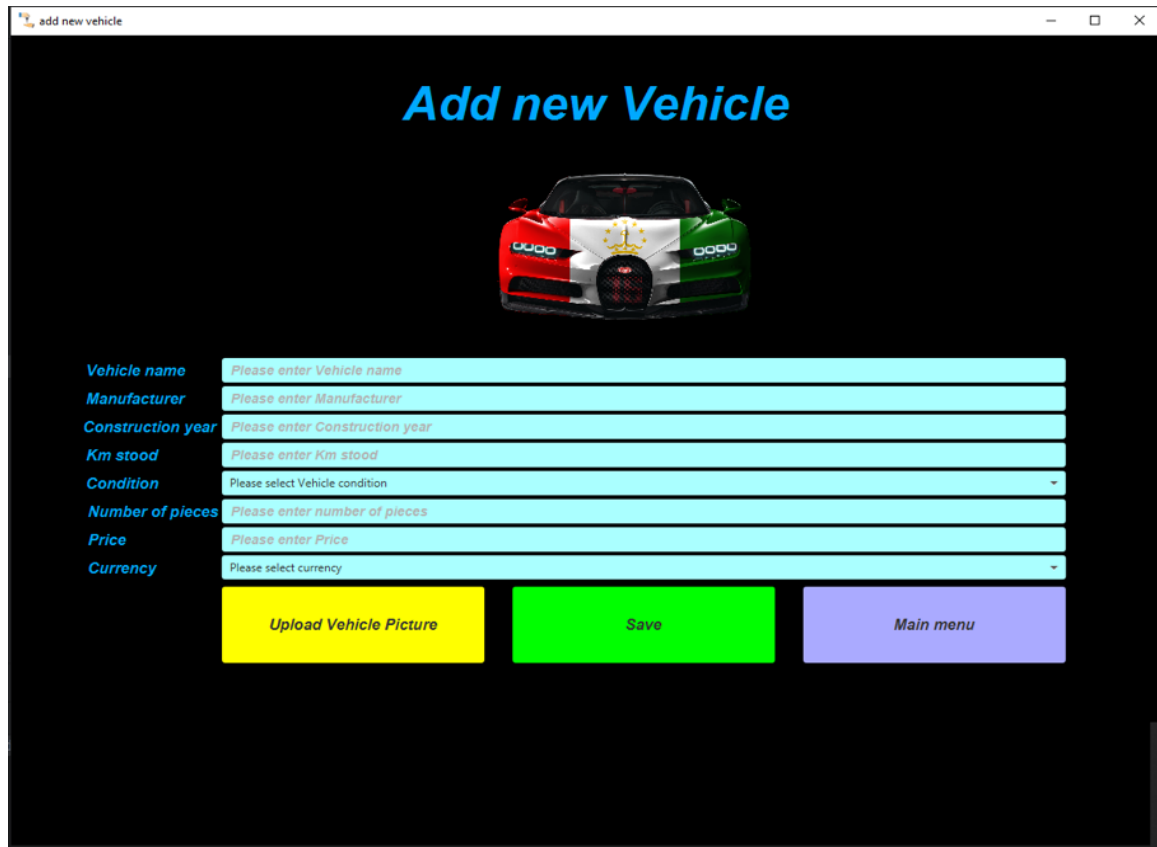


Il comprend 3 Menu qui inclus 1 menubar puis 3 menuitem où vous aller inclure une photo au format w :40 h :40

Question 5 : créer la scène d'ajout d'un véhicule + Layout + ID Node

Dans le fichier fxml correspondant vous devez faire pointer votre contrôleur (fx:controller) sur la classe NewVehicle.

Puis implémenter via Scene Builder l'interface suivante :



Vous devez également renseigner les ID Node dans Scene Builder (dans votre layout), c'est-à-dire renseigner le champs fx:id dans la partie code (à droite de l'écran) avec comme nomenclature tf_nomchamp pour textfield, button_nombouton pour un bouton et pour votre imageview (imageview_nomimage).

Question 6 : créer la fonction MenuItemem dans le HelloControlers. Elle vous permet de faire apparaître votre scene AddNewVehicle dans votre fenêtre.

Tout d'abord, vous devez implementer la class Initializable sur la classe Hellocontroler pour avoir accès aux fonctionnalités JavaFx. Le terme sera souligné vous devez faire clique gauche puis implements methods.

Déclarer votre objet MenuItemem_ qui correspond à votre addNewVehicle comme ceci :

```
@FXML
public MenuItem menuitem_addnewvehicule;
```

Puis dans la méthode public void initialize, vous devez créer l'action de votre bouton :

```
menuitem_addnewvehicule.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        FXMLLoader loader = new FXMLLoader(AddNewVehicle.class.getResource("addnewvehicle.fxml"));
        try {
            Parent root = loader.load();
            Stage stage = (Stage) mymenubar.getScene().getWindow();
            stage.setScene(new Scene(root, 1200, 900));
            stage.setMaxHeight(900);
            stage.setMaxWidth(1200);
            stage.setTitle("add new vehicle");
            stage.show();
        }
    }
});
```

```

        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
});

```

Plus concrètement, cette méthode permet de créer un événement lorsque le bouton est actionné. Elle va alors charger votre fichier FXML passé en paramètre et créer une nouvelle scene à l'intérieur de votre fenêtre principale.

Testez alors que votre fenêtre add new vehicle apparaît bien lorsque vous cliquez dans le menu bar sur add new vehicle

Question 7 : créer la connexion à la base de donnée

Créer une classe DBConnection avec le code suivant pour vous connecter :

```

public class DBConnection {
    public static Connection getConnection() throws SQLException {
        try {
            // Charge le pilote JDBC MySQL
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
            throw new SQLException("MySQL JDBC driver not found", e);
        }
        // Retourne la connexion établie à la base de données
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/votre_bdd", "root", "votre_mdp"); /*renseig
    }
}

```

Vérifiez que le mysql-connectorj est dans un répertoire de votre projet et importer en tant que jar. (Clique droit "Add as Library sur votre répertoire.

Question 8 : créer l'upload de l'image

Dans votre fichier add new vehicle, importer les données du fichier FXML. Le principe du bouton sera le suivant :

1. Lorsque l'utilisateur clique sur un bouton intitulé "Upload Vehicle ", une fenêtre de dialogue de sélection de fichier doit s'ouvrir.
2. Cette fenêtre de dialogue doit être configurée pour filtrer les fichiers uniquement avec les extensions suivantes : **.jpg**, **.png**, **.jpeg** et **.svg**.
3. Une fois que l'utilisateur sélectionne un fichier image valide et clique sur "Ouvrir", vous devez récupérer le chemin absolu du fichier sélectionné.
4. Affichez le chemin absolu du fichier sélectionné dans la console.
5. Chargez l'image sélectionnée à l'aide de ce chemin absolu et affichez-la dans une vue d'image prévue à cet effet sur l'interface utilisateur.

Pour ce faire, vous pouvez rajouter ce code dans votre page en ayant déclaré au préalable les variables.

```
/*@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@  
@@@@@@@@@@@@*/  
button_upload_picture.setOnAction(new EventHandler() {  
    @Override  
    public void handle(ActionEvent event) {  
        FileChooser fileChooser = new FileChooser();  
        fileChooser.setTitle("Upload Vehicle Picture");  
        fileChooser.getExtensionFilters().addAll(new  
FileChooser.ExtensionFilter("Image Files", "*.jpg", "*.png", "*.jpeg",  
"*.*svg"));  
        Stage stage = (Stage) myanchorpane.getScene().getWindow();  
  
        File file = fileChooser.showOpenDialog(null);  
        if (file == null) {  
            return;  
        } else {  
            try {  
                path = file.getAbsolutePath();  
                System.out.println(path);  
                Image image = new Image(new FileInputStream(path));  
                imageView_add_new_vehicle.setImage(image);  
            } catch (FileNotFoundException e) {  
                throw new RuntimeException(e);  
            }  
        }  
    }  
});
```

Question 9 : Créer la fonction de sauvegarde

1. Lorsque l'utilisateur clique sur un bouton intitulé "Save", les détails du véhicule saisis dans les champs de texte et les menus déroulants doivent être récupérés.
2. Assurez-vous que tous les champs obligatoires sont remplis. S'il y a des champs vides, affichez une boîte de dialogue d'erreur indiquant que les champs d'entrée ou de sélection ne peuvent pas être vides, puis arrêtez le traitement.
3. Convertissez les valeurs numériques entrées pour le kilométrage, le prix et le nombre de pièces en leurs types respectifs (`double` et `int`).
4. Ouvrez un flux d'entrée pour lire l'image sélectionnée à l'aide du chemin du fichier.
5. Établissez une connexion à une base de données pour enregistrer les détails du véhicule, y compris l'image, dans une table appropriée.
6. Insérez les détails du véhicule dans la base de données à l'aide d'une requête SQL préparée.

```

1. /*@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@*/
2. button_save.setOnAction(new EventHandler<ActionEvent>() {
3.     @Override
4.     public void handle(ActionEvent event) {
5.         // Récupération des détails du véhicule à partir des champs de texte et des menus déroulants
6.         input_vehicle_name = tf_vehicle_name.getText();
7.         ...
8.
9.         // Vérification que tous les champs obligatoires sont remplis
10.        if(vos champs vides) {
11.            Alert alert = new Alert(Alert.AlertType.ERROR);
12.            alert.setTitle("Input error");
13.            alert.setContentText("Input or select fields cannot be empty");
14.            alert.show();
15.            return;
16.        } else {
17.            // Conversion des valeurs numériques en types appropriés
18.            input_kmStood = Double.parseDouble(km_stood);
19.            input_price = Double.parseDouble(price);
20.            input_pieces = Integer.parseInt(pieces);
21.
22.
23.

```

```

28.         try {
29.             // Ouverture d'un flux d'entrée pour lire l'image sélectionnée
30.             FileInputStream fileInputStream = new FileInputStream(path);
31.
32.             // Établissement d'une connexion à la base de données
33.             Connection con = DBConnection.getConnection();
34.
35.             // Préparation de la requête SQL pour l'insertion des détails du véhicule
36.             pre = con.prepareStatement("INSERT INTO vehicle (vos champs )VALUES
(?)?,?,?,?,?,?,?,?)");
37.             pre.setString(1, input_vehicle_name);
38.             pre.setString(2, input_manufacturer);
39.             pre.setString(3, input_construction_year);
40.             pre.setDouble(4, input_kmStood);
41.             pre.setString(5, selection_condition);
42.             pre.setInt(6, input_pieces);
43.             pre.setDouble(7, input_price);
44.             pre.setString(8, select_currency);
45.             pre.setBlob(9, fileInputStream);
46.
47.             // Exécution de la requête SQL pour insérer les détails du véhicule dans la
base de données
48.             pre.executeUpdate();
49.
50.             // Chargement de la nouvelle interface utilisateur affichant une table de
véhicules
51.             FXMLLoader loader = new
FXMLLoader(VehicleTable.class.getResource("vehicletable.fxml"));
52.             Parent root = loader.load();
53.             Stage stage =(Stage) button_save.getScene().getWindow();
54.             stage.setScene(new Scene(root, 1200, 900));
55.             stage.setMaxWidth(1200);
56.             stage.setMaxHeight(900);
57.             stage.setTitle("vehicle Table");
58.             stage.show();
59.
60.         } catch (SQLException | IOException e) {
61.             throw new RuntimeException(e);
62.         }
63.     }
64.

```

Question 10 : Créer le retour Main menu

1. Lorsque l'utilisateur clique sur un bouton intitulé "Main Menu", l'application doit charger l'interface utilisateur du menu principal.
2. L'interface utilisateur du menu principal est définie dans le fichier **hello-view.fxml**.
3. Assurez-vous que la nouvelle interface utilisateur est chargée dans la fenêtre principale de l'application.

Pour vous aider :

```

FXMLLoader loader = new
FXMLLoader(HelloController.class.getResource("hello-view.fxml"));
Parent root = loader.load();
Stage stage = (Stage) button_main_menu.getScene().getWindow();

```

Question 11 : Créer la page avec le Tableview pour la table vehicle



Nommer votre item puis rajouter le dans votre hello controller (voir Q6)

Créer le layout vehicle table et son controller.



Il faut nommer un fx id sur chaque colonne de la tableview et la tableview puis importer dans le controller.

Maintenant, on va relier les données à la tableView.

1. Créer le modèle (classe voiture) dans un nouveau fichier ModelTableVehicle.class avec le constructeur et les get et set et ces 10 objets.
2. Importer toutes les variables du fichier FXML correspondant au Fx :id (tableView, tableColumn et bouton)

Vous devez vous appuyer sur votre modèle voiture.

Cela devra ressembler à ce code :

```
1. Public TableColumn<Nom_De_votre_Classe,type> nom_de_votre colonne
```

3. Créer d'autres variable : PreparedStatement pre (pour vos requêtes préparées sur la base de données), ResultSet rs (pour récupérer vos resultats de requêtes), une liste sous forme de tableau Observable<nom_de_votre_classe> obs = FXCollections.observableArrayList()
Des public static String, integer, double et un Blob avec les différents éléments de votre base de données.

4. Dans votre méthode initialize, vous allez copier le code suivant pour faire afficher vos véhicules :

```
5. @Override
6.     public void initialize(URL url, ResourceBundle resourceBundle) {
7. // commentaire sur ce que fais le code suivant :
8.         tablecolumn_id.setStyle("-fx-background-color:black; -fx-text-
          fill:#00aaff;-fx-font-size:16");
9.         tablecolumn_vehicle_name.setStyle("-fx-background-color:black; -fx-
          text-fill:#00aaff;-fx-font-size:16");
10.        tablecolumn_manufacturer.setStyle("-fx-background-color:black; -fx-
          text-fill:#00aaff;-fx-font-size:16");
```

```
11.         tablecolumn_construction_year.setStyle("-fx-background-color:black;
-fx-text-fill:#00aaff;-fx-font-size:16");
12.         tablecolumn_condition.setStyle("-fx-background-color:black; -fx-
text-fill:#00aaff;-fx-font-size:16");
13.         tablecolumn_km_stood.setStyle("-fx-background-color:black; -fx-
text-fill:#00aaff;-fx-font-size:16");
14.         tablecolumn_price.setStyle("-fx-background-color:black; -fx-text-
fill:#00aaff;-fx-font-size:16");
15.         tablecolumn_pieces.setStyle("-fx-background-color:black; -fx-text-
fill:#00aaff;-fx-font-size:16");
16.         tablecolumn_currency.setStyle("-fx-background-color:black; -fx-
text-fill:#00aaff;-fx-font-size:16");
17.         tablecolumn_availability.setStyle("-fx-background-color:black; -fx-
text-fill:#00aaff;-fx-font-size:16");
18.
19.         System.out.println(observableList);
20.
21.         try {
22.             Connection con = DBConnection.getConnection();
23.             pre = con.prepareStatement(" requête à effectuer");
24.             rs = pre.executeQuery();
25. // que fait cette partie ?
26.             while (rs.next()){
27.                 observableList.add(new
ModelTableVehicle(rs.getString("vehicle_id"), rs.getString("vehicle_name"),
rs.getString("manufacturer"), rs.getString("construction_year"),
rs.getString("km_stood"), rs.getString("vehicle_condition"),
rs.getString("pieces"), rs.getString("price"), rs.getString("currency"),
rs.getString("availability")));
28.             }
29.             System.out.println(observableList);
30.         } catch (SQLException e) {
31.             throw new RuntimeException(e);
32.         }
33. // que fait cette partie ?
34.         tablecolumn_id.setCellValueFactory(new
PropertyValueFactory<>("vehicle_id"));
35.         tablecolumn_vehicle_name.setCellValueFactory(new
PropertyValueFactory<>("vehicle_name"));
36.         tablecolumn_manufacturer.setCellValueFactory(new
PropertyValueFactory<>("manufacturer"));
37.         tablecolumn_construction_year.setCellValueFactory(new
PropertyValueFactory<>("construction_year"));
38.         tablecolumn_km_stood.setCellValueFactory(new
PropertyValueFactory<>("km_stood"));
39.         tablecolumn_condition.setCellValueFactory(new
PropertyValueFactory<>("vehicle_condition"));
40.         tablecolumn_pieces.setCellValueFactory(new
PropertyValueFactory<>("pieces"));
41.         tablecolumn_price.setCellValueFactory(new
PropertyValueFactory<>("price"));
42.         tablecolumn_currency.setCellValueFactory(new
PropertyValueFactory<>("currency"));
43.         tablecolumn_availability.setCellValueFactory(new
PropertyValueFactory<>("availability"));
```

```
tableView_vehicle.setItems(observableList);
```

Vous devez compléter la requête et commenter les zones de code.

Question 12 : Créer la fonction du clique de souris sur VehicleTable pour faire apparaitre votre véhicule dans un nouveau layout avec toutes ses données.

Créer un fichier FXML, nommé Selectedvehicle.fxml que vous laissez vide pour le moment.

Utilisez le code fourni que vous complétez et commenterez.

[illegible]

Question 13 : Créer la scène qui apparaîtra lorsque vous aurez sélectionné le véhicule :

[illegible]

```
tf_availability.setText(availability);
```

Question 16 : Créer la fonction `upload picture` et commenter le code

```

/*
 *
 */
button_upload_picture.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Upload Vehicle Picture");
        fileChooser.getExtensionFilters().addAll(new
FileChooser.ExtensionFilter("Image Files", "*.jpg", "*.png", "*.jpeg",
"*.svg"));

        Stage stage = (Stage) myanchorpane.getScene().getWindow();

        File file = fileChooser.showOpenDialog(null);
        if (file == null) {
            return;
        } else {
            try {
                path = file.getAbsolutePath();
                System.out.println(path);
                Image image = new Image(new FileInputStream(path));
                imageview_vehicle_picture.setImage(image);
            } catch (FileNotFoundException e) {
                throw new RuntimeException(e);
            }
        }
    }
}
}

```

Question 17 : créer l'update du la photo

```

button_save_picture.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        if(path==null){
            Alert alert = new Alert(Alert.AlertType.ERROR);
            alert.setContentText("Please select Vehicle picture file
!");

            alert.setTitle("Warning");
            alert.show();
        }else{
            try {
                Connection con = DBConnection.getConnection();
                FileInputStream fileInputStream = new
FileInputStream(path);

                pre = con.prepareStatement("à remplir");
                pre.setBlob(1,fileInputStream);
                pre.executeUpdate();
            } catch (SQLException | FileNotFoundException e) {
                throw new RuntimeException(e);
            }
        }
    }
}

```

```
});
```

Question 18 : Créer la fonction delete et TableVehicule

Pour le bouton delete

[illegible]

Pour le tablevehicule, vous devez simplement appeler votre fichier FXML correspondant.

A vous de jouer avec ce que vous avez vu tout au long du tp

Question 19 : créer la fonction update vehicle et commenter les différentes parties

```
button_update.setOnAction(new EventHandler<ActionEvent>() {
    @Override
    public void handle(ActionEvent event) {
        try {
            Connection con = DBConnection.getConnection();
            pre = con.prepareStatement("à compléter");
            String name=tf_vehicle_name.getText();
            String manufacturer = tf_manufacturer.getText();
```

```

        String construction_year = tf_construction_year.getText();
        Double km_stood = Double.valueOf(tf_km_stood.getText());
        String condition = tf_condition.getText();
        Integer pieces = Integer.valueOf(tf_pieces.getText());
        Double price = Double.valueOf(tf_price.getText());
        String currency = tf_currency.getText();
        String availability = tf_availability.getText();

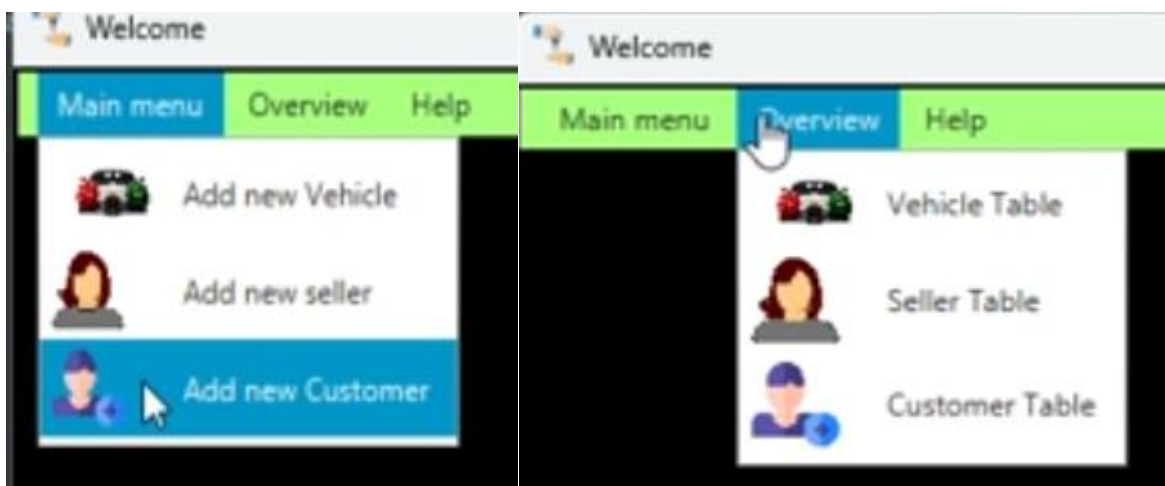
        pre.setString(1,name);
        pre.setString(2, manufacturer);
        pre.setString(3,construction_year);
        pre.setDouble(4,(km_stood));
        pre.setString(5,condition);
        pre.setInt(6, (pieces));
        pre.setDouble(7, (price));
        pre.setString(8,currency);
        pre.setString(9,availability);

        pre.executeUpdate();
        FXMLLoader loader = new
FXMLLoader(VehicleTable.class.getResource("vehicletable.fxml"));
        Parent root = loader.load();
        Stage stage = (Stage)
button_update.getScene().getWindow();
        stage.setScene(new Scene(root, 1200,900));
        stage.setMaxWidth(1200);
        stage.setMaxHeight(900);
        stage.setTitle("vehicle table");
        stage.show();

    } catch (SQLException e) {
        throw new RuntimeException(e);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
});

```

Question 20 : Reproduire la même chose pour les vendeurs et les acheteurs selon les écrans suivants :



Pour la partie seller :

Add new seller



Seller name

Seller Phone

Seller E-mail

Seller Address

Upload seller picture **Save** **Main menu**

Seller Table

ID	Name	Phone	Email	Address
1	Kate Hanz	+142536987	kate.hanz@gmail.com	Str 12 70125 City

Main menu

Selected Seller

Update Seller Data

ID

Name

Phone

E-Mail

Address

Upload picture **Save picture** **Update Seller Data** **Delete Seller Data** **Table Seller**

Pour la partie customer :

Add New Customer

Add new Customer



Name

Phone

E-Mail

Address

Please enter Customer Phone

Please enter Customer E-mail

Please enter Customer Address

Save

Main menu

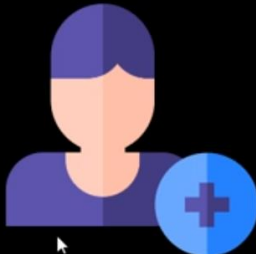
Customer Table

ID	Name	Phone	E-Mail	Address
1	Kate mann	+125463987	kate.mann@gmail.com	Muster str 23 145236 MusterCity
2	Felex Frank	+125639844	felex.frank@gmail.com	Muster str 5 785692 MusterCity

Main menu

Selected Customer

Selected Customer



ID

Name

Phone

E-Mail

Address

1

Kate mann

+125463987

kate.mann@gmail.com

Muster str 23 145236 MusterCity

Update Customer Data

Delete Customer Data

Customer Table